

Vježba - Theme Preferences aplikacija pomoću custom hooka

useLocalStorage

U ovoj vježbi izradit ćemo malu aplikaciju u kojoj korisnik može odabrati temu sučelja. Aplikacija će imati **light** i **dark** temu, a odabrana tema će ostati zapamćena čak i nakon refreshanja stranice.

Drugim riječima, ako korisnik odabere tamnu temu i zatim osvježi browser, aplikacija se neće vratiti na početnu svijetlu temu. Umjesto toga, React će pročitati spremljenu vrijednost iz `localStorage`.

Ova vježba je dobar primjer za custom hook zato što se isti obrazac može koristiti na više mjesta:

- state vrijednost
- čitanje početne vrijednosti iz `localStorage`a
- spremanje vrijednosti u `localStorage` kada se promijeni

Umjesto da tu logiku pišemo direktno u `App` komponenti, izdvojit ćemo je u custom hook.

Cilj vježbe

Cilj je napraviti custom hook:

`useLocalStorage(key, initial)`

koji se ponaša slično kao `useState`, ali dodatno sinkronizira vrijednost s `localStorageom`.

Na kraju vježbe aplikacija treba omogućiti korisniku da:

- prebacuje između light i dark teme
- vidi trenutnu temu
- vidi mali preview kartice u odabranoj temi
- refreshanjem stranice zadrži odabranu temu

Naziv projekta

08_React/15_Practice_ThemePreferences/

Što ćemo koristiti

U ovoj vježbi koristimo koncepte koje smo već prošli:

komponente
useState
useEffect
custom hook
controlled behavior preko statea
conditional rendering
event handlers
CSS klase
localStorage

Korak 1 — Kreiranje projekta

U terminalu kreirajte novi Vite React projekt:

```
npm create vite@latest theme-preferences -- --template react
```

Uđite u projekt:

```
cd theme-preferences
```

Instalirajte dependencyje:

```
npm install
```

Pokrenite aplikaciju:

```
npm run dev
```

Korak 2 — Očistite početni projekt

U `src/App.jsx` obrišite početni sadržaj i pripremite osnovnu komponentu:

```
import './App.css';
```

```
function App() {  
  return (  
    <main className="app">  
      <h1>Theme Preferences</h1>  
    </main>  
  );  
}
```

export default App;

U `src/App.css` obrišite postojeći CSS.

Korak 3 — Napravite folder za custom hookove

U `src` folderu napravite novi folder:

hooks

Zatim u njemu napravite datoteku:

useLocalStorage.js

Struktura sada može izgledati ovako:

```
src/  
  App.jsx  
  App.css  
  main.jsx  
  hooks/  
    useLocalStorage.js
```

Ova organizacija nije obavezna, ali je dobra navika. Kada aplikacija raste, custom hookove je lakše pronaći ako su u posebnom folderu.

Korak 4 — Napravite osnovni custom hook

U datoteku `src/hooks/useLocalStorage.js` prvo dodajte import:

```
import { useEffect, useState } from "react";
```

Zatim napravite funkciju:

```
function useLocalStorage(key, initial) {  
  
}
```

```
export default useLocalStorage;
```

Važno je da ime hooka počinje s `use`.

Ovo nije samo stil pisanja. React i linting alati očekuju da funkcije koje koriste hookove imaju `use` prefiks.

Dobro:

```
function useLocalStorage() {  
}
```

Loše:

```
function localStorageState() {  
}
```

Ako funkcija koristi `useState` ili `useEffect`, treba biti custom hook i treba početi s `use`.

Korak 5 — Dodajte state u custom hook

Unutar hooka dodajte običan `useState`.

Za početak ćemo napraviti najjednostavniju verziju:

```
import { useEffect, useState } from "react";
```

```
function useLocalStorage(key, initial) {  
  const [value, setValue] = useState(initial);  
  
  return [value, setValue];  
}
```

```
export default useLocalStorage;
```

Ova verzija se zasad ponaša kao obični `useState`.

Prima početnu vrijednost:

initial

i vraća:

[value, setValue]

Zašto vraćamo array?

Zato što želimo da se hook koristi slično kao `useState`.

Kasnije ćemo u komponenti moći pisati:

```
const [theme, setTheme] = useLocalStorage("theme", "light");
```

To izgleda vrlo poznato jer podsjeća na:

```
const [theme, setTheme] = useState("light");
```

Korak 6 — Koristite hook u **App** komponenti

U `src/App.jsx` importajte hook:

```
import useLocalStorage from "../hooks/useLocalStorage";  
import "../App.css";
```

Zatim ga koristite unutar **App** komponente:

```
function App() {  
  const [theme, setTheme] = useLocalStorage("theme", "light");  
  
  return (  
    <main className="app">  
      <h1>Theme Preferences</h1>  
  
      <p>Current theme: {theme}</p>  
  
      <button onClick={() => setTheme("dark")}>  
        Use dark theme  
      </button>  
  
      <button onClick={() => setTheme("light")}>  
        Use light theme
```

```
    </button>
  </main>
);
}
```

Cijeli `App.jsx` sada izgleda ovako:

```
import useLocalStorage from "../hooks/useLocalStorage";
import "../App.css";

function App() {
  const [theme, setTheme] = useLocalStorage("theme", "light");

  return (
    <main className="app">
      <h1>Theme Preferences</h1>

      <p>Current theme: {theme}</p>

      <div className="actions">
        <button onClick={() => setTheme("dark")}>
          Use dark theme
        </button>

        <button onClick={() => setTheme("light")}>
          Use light theme
        </button>
      </div>
    </main>
  );
}

export default App;
```

U ovom trenutku tema se mijenja, ali još se ne sprema u `localStorage`.

Korak 7 — Spremite vrijednost u `localStorage`

Sada ćemo se vratiti u `useLocalStorage.js`.

Želimo da se svaki put kada se `value` promijeni, nova vrijednost spremi u `localStorage`.

Za to koristimo `useEffect`.

Dodajte:

```
useEffect(() => {  
  localStorage.setItem(key, JSON.stringify(value));  
}, [key, value]);
```

Cijeli hook sada izgleda ovako:

```
import { useEffect, useState } from "react";  
  
function useLocalStorage(key, initial) {  
  const [value, setValue] = useState(initial);  
  
  useEffect(() => {  
    localStorage.setItem(key, JSON.stringify(value));  
  }, [key, value]);  
  
  return [value, setValue];  
}
```

export default useLocalStorage;

Ovdje radimo dvije važne stvari.

Prvo, spremamo vrijednost pod ključem:

key

Ako pozovemo hook ovako:

```
useLocalStorage("theme", "light");
```

vrijednost će biti spremljena u `localStorage` pod ključem `"theme"`.

Drugo, koristimo:

```
JSON.stringify(value)
```

`localStorage` sprema vrijednosti kao stringove. Zato vrijednost pretvaramo u JSON string.

To znači da hook neće raditi samo za stringove poput `"dark"`, nego kasnije može raditi i za arrayeve ili objekte.

Korak 8 — Pročitajte početnu vrijednost iz `localStorage`a

Trenutno aplikacija sprema vrijednost u `localStorage`, ali ima problem.

Ako refreshamo stranicu, `useState(initial)` ponovno postavlja vrijednost na `"light"`.

Zato moramo početnu vrijednost pročitati iz `localStorage`a.

Umjesto ovoga:

```
const [value, setValue] = useState(initial);
```

napraviti ćemo ovo:

```
const [value, setValue] = useState(() => {  
  const raw = localStorage.getItem(key);
```

```
  if (raw === null) {  
    return initial;  
  }
```

```
  return JSON.parse(raw);  
});
```

Cijeli hook sada izgleda ovako:

```
import { useEffect, useState } from "react";
```

```
function useLocalStorage(key, initial) {  
  const [value, setValue] = useState(() => {  
    const raw = localStorage.getItem(key);
```

```
    if (raw === null) {  
      return initial;  
    }
```

```
    return JSON.parse(raw);  
  });
```

```
  useEffect(() => {  
    localStorage.setItem(key, JSON.stringify(value));  
  }, [key, value]);
```

```
  return [value, setValue];  
}
```



```
export default useLocalStorage;
```

Ovdje koristimo posebnu verziju `useState` početne vrijednosti.

Umjesto da odmah predamo vrijednost:

```
useState(initial)
```

predajemo funkciju:

```
useState(() => {  
  // read localStorage  
})
```

Ta funkcija se zove **initializer function**.

React je poziva samo jednom, kod prvog renderiranja komponente.

To je važno jer ne želimo čitati `localStorage` pri svakom renderiranju. Dovoljno je jednom, kada komponenta prvi put treba početnu vrijednost.

Korak 9 — Dodajte toggle funkciju

Sada u `App.jsx` možemo napraviti funkciju koja prebacuje temu iz `light` u `dark` i obrnuto.

Dodajte unutar `App` komponente:

```
function toggleTheme() {  
  setTheme(currentTheme =>  
    currentTheme === "light" ? "dark" : "light"  
  );  
}
```

Ovdje koristimo functional updater oblik:

```
setTheme(currentTheme => ...)
```

To je dobar obrazac kada nova vrijednost ovisi o prethodnoj vrijednosti.

Sada možemo zamijeniti dva gumba jednim gumbom:

```
<button onClick={toggleTheme}>  
  Switch to {theme === "light" ? "dark" : "light"} theme  
</button>
```

Korak 10 — Primijenite temu na aplikaciju

U `main` element dodajte klasu koja ovisi o trenutnoj temi:

```
<main className={`app ${theme === "dark" ? "dark" : "light"}}`>
```

Cijeli `App.jsx` sada izgleda ovako:

```
import useLocalStorage from "../hooks/useLocalStorage";
import "../App.css";

function App() {
  const [theme, setTheme] = useLocalStorage("theme", "light");

  function toggleTheme() {
    setTheme(currentTheme =>
      currentTheme === "light" ? "dark" : "light"
    );
  }

  return (
    <main className={`app ${theme === "dark" ? "dark" : "light"}}`>
      <section className="panel">
        <p className="eyebrow">Custom Hooks Practice</p>

        <h1>Theme Preferences</h1>

        <p>
          Choose your preferred theme. The application will remember your
          choice even after you refresh the page.
        </p>

        <div className="theme-status">
          Current theme: <strong>{theme}</strong>
        </div>

        <button onClick={toggleTheme}>
          Switch to {theme === "light" ? "dark" : "light"} theme
        </button>

        <article className="preview-card">
          <h2>Preview Card</h2>
          <p>
            This card changes together with the selected theme. The value is
            stored in localStorage through a reusable custom hook.
          </p>
        </article>
      </section>
    </main>
  );
}
```

```
    </p>
  </article>
</section>
</main>
);
}
```

export default App;

Ovo je sada mala, stvarna aplikacija. Nije samo hook u izolaciji.

Korisnik vidi trenutnu temu, može je promijeniti, a UI se mijenja na temelju spremljene vrijednosti.

Korak 11 — Dodajte CSS

U `src/App.css` dodajte:

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
}
```

```
.app {
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 32px;
  box-sizing: border-box;
}
```

```
.app.light {
  background-color: #eef2f7;
  color: #1f2937;
}
```

```
.app.dark {
  background-color: #111827;
  color: #f9fafb;
}
```

```
.panel {
  width: 100%;
  max-width: 560px;
```

```
padding: 32px;  
border-radius: 18px;  
}
```

```
.app.light .panel {  
background-color: white;  
border: 1px solid #d8dee9;  
}
```

```
.app.dark .panel {  
background-color: #1f2937;  
border: 1px solid #374151;  
}
```

```
.eyebrow {  
margin: 0 0 8px;  
text-transform: uppercase;  
letter-spacing: 0.08em;  
font-size: 13px;  
font-weight: bold;  
opacity: 0.75;  
}
```

```
h1 {  
margin-top: 0;  
font-size: 36px;  
}
```

```
p {  
line-height: 1.6;  
}
```

```
.theme-status {  
margin: 24px 0;  
padding: 12px 14px;  
border-radius: 10px;  
}
```

```
.app.light .theme-status {  
background-color: #f2f4f7;  
}
```

```
.app.dark .theme-status {  
background-color: #374151;  
}
```

```
button {  
padding: 10px 16px;
```

```
border: none;
border-radius: 8px;
cursor: pointer;
font-size: 15px;
}

.app.light button {
background-color: #1f2937;
color: white;
}

.app.dark button {
background-color: #f9fafb;
color: #111827;
}

.preview-card {
margin-top: 24px;
padding: 20px;
border-radius: 14px;
}

.app.light .preview-card {
background-color: #f8fafc;
border: 1px solid #d8dee9;
}

.app.dark .preview-card {
background-color: #111827;
border: 1px solid #4b5563;
}

.preview-card h2 {
margin-top: 0;
}
```

Korak 12 — Testirajte aplikaciju

Sada testirajte ponašanje.

Prvo otvorite aplikaciju.

Trebali biste vidjeti početnu temu:

light

Kliknite gumb.

Tema se treba promijeniti u:

dark

Sada refreshajte browser.

Ako je sve dobro napravljeno, aplikacija treba ostati u dark temi.

Zatim ponovno kliknite gumb da se tema vrati na:

light

Ponovno refreshajte browser.

Aplikacija treba ostati u light temi.

To znači da custom hook uspješno radi.

Finalno rješenje

src/hooks/useLocalStorage.js

```
import { useEffect, useState } from "react";

function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const raw = localStorage.getItem(key);

    if (raw === null) {
      return initial;
    }

    return JSON.parse(raw);
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

export default useLocalStorage;
```

src/App.jsx

```
import useLocalStorage from "../hooks/useLocalStorage";
import "../App.css";

function App() {
  const [theme, setTheme] = useLocalStorage("theme", "light");

  function toggleTheme() {
    setTheme(currentTheme =>
      currentTheme === "light" ? "dark" : "light"
    );
  }

  return (
    <main className={`app ${theme === "dark" ? "dark" : "light"}`} >
      <section className="panel">
        <p className="eyebrow">Custom Hooks Practice</p>

        <h1>Theme Preferences</h1>

        <p>
          Choose your preferred theme. The application will remember your
          choice even after you refresh the page.
        </p>

        <div className="theme-status">
          Current theme: <strong>{theme}</strong>
        </div>

        <button onClick={toggleTheme}>
          Switch to {theme === "light" ? "dark" : "light"} theme
        </button>

        <article className="preview-card">
          <h2>Preview Card</h2>
          <p>
            This card changes together with the selected theme. The value is
            stored in localStorage through a reusable custom hook.
          </p>
        </article>
      </section>
    </main>
  );
}
```

export default App;

src/App.css

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
}
```

```
.app {  
  min-height: 100vh;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  padding: 32px;  
  box-sizing: border-box;  
}
```

```
.app.light {  
  background-color: #eef2f7;  
  color: #1f2937;  
}
```

```
.app.dark {  
  background-color: #111827;  
  color: #f9fafb;  
}
```

```
.panel {  
  width: 100%;  
  max-width: 560px;  
  padding: 32px;  
  border-radius: 18px;  
}
```

```
.app.light .panel {  
  background-color: white;  
  border: 1px solid #d8dee9;  
}
```

```
.app.dark .panel {  
  background-color: #1f2937;  
  border: 1px solid #374151;  
}
```



```
.eyebrow {  
  margin: 0 0 8px;  
  text-transform: uppercase;  
  letter-spacing: 0.08em;  
  font-size: 13px;  
  font-weight: bold;  
  opacity: 0.75;  
}
```

```
h1 {  
  margin-top: 0;  
  font-size: 36px;  
}
```

```
p {  
  line-height: 1.6;  
}
```

```
.theme-status {  
  margin: 24px 0;  
  padding: 12px 14px;  
  border-radius: 10px;  
}
```

```
.app.light .theme-status {  
  background-color: #f2f4f7;  
}
```

```
.app.dark .theme-status {  
  background-color: #374151;  
}
```

```
button {  
  padding: 10px 16px;  
  border: none;  
  border-radius: 8px;  
  cursor: pointer;  
  font-size: 15px;  
}
```

```
.app.light button {  
  background-color: #1f2937;  
  color: white;  
}
```

```
.app.dark button {  
  background-color: #f9fafb;  
  color: #111827;
```

```
}

.preview-card {
  margin-top: 24px;
  padding: 20px;
  border-radius: 14px;
}

.app.light .preview-card {
  background-color: #f8fafc;
  border: 1px solid #d8dee9;
}

.app.dark .preview-card {
  background-color: #111827;
  border: 1px solid #4b5563;
}

.preview-card h2 {
  margin-top: 0;
}
```

Objašnjenje rješenja

Najvažniji dio ove vježbe je custom hook:

```
function useLocalStorage(key, initial) {
```

Hook prima dva argumenta.

key je naziv pod kojim spremamo vrijednost u **localStorage**.

Primjer:

```
useLocalStorage("theme", "light");
```

Ovdje je **"theme"** ključ.

initial je početna vrijednost ako u **localStorageu** još ništa nije spremljeno.

Unutar hooka koristimo:

```
const [value, setValue] = useState(() => {
```

Ovdje ne predajemo običnu vrijednost u `useState`, nego funkciju.

To radimo zato što čitanje iz `localStoragea` želimo napraviti samo jednom, prilikom prvog renderiranja.

Da smo napisali ovako:

```
const raw = localStorage.getItem(key);  
const [value, setValue] = useState(...);
```

čitanje bi se izvršilo svaki put kada se hook pozove tijekom renderiranja.

Initializer funkcija je bolja jer React ovu funkciju koristi samo za početnu vrijednost.

Ovaj dio čita spremljenu vrijednost:

```
const raw = localStorage.getItem(key);
```

Ako ništa ne postoji u `localStorageu`, `raw` će biti `null`.

Zato provjeravamo:

```
if (raw === null) {  
  return initial;  
}
```

Ako vrijednost postoji, vraćamo:

```
return JSON.parse(raw);
```

Zato što smo podatak spremili kao JSON string, moramo ga pročitati natrag pomoću `JSON.parse`.

Drugi važan dio je `useEffect`:

```
useEffect(() => {  
  localStorage.setItem(key, JSON.stringify(value));  
}, [key, value]);
```

Ovaj effect se izvršava svaki put kada se promijeni `key` ili `value`.

U našem primjeru `key` ostaje `"theme"`, ali `value` se mijenja između:

```
light  
dark
```

Kada se `value` promijeni, effect spremi novu vrijednost u `localStorage`.

Koristimo:

```
JSON.stringify(value)
```

jer `localStorage` sprema samo stringove.

Na kraju hook vraća:

```
return [value, setValue];
```

Ovo je namjerno isto kao kod `useState`.

Zato možemo pisati:

```
const [theme, setTheme] = useLocalStorage("theme", "light");
```

Ovaj custom hook postaje gotovo zamjena za `useState`, ali s dodatnom sposobnošću spremanja u `localStorage`.

To je jako važan obrazac kod custom hookova: komponenta dobiva jednostavno sučelje, a kompleksnost ostaje skrivena unutar hooka.

Zašto je ovo custom hook, a ne komponenta?

Ovo je custom hook zato što ne vraća JSX.

Hook ne odlučuje kako će aplikacija izgledati.

Hook samo upravlja ponašanjem:

čita vrijednost
sprema vrijednost
vraća state i setter funkciju

Komponenta `App` odlučuje kako će UI izgledati:

naslov
gumb
kartica
boje

tekstovi

To je glavna razlika:

custom hook = ponašanje i logika

komponenta = korisničko sučelje

Dodatni izazovi

Izazov 1: Spremite ime korisnika

Dodajte još jedan `useLocalStorage` poziv:

```
const [name, setName] = useLocalStorage("name", "");
```

Dodajte input u kojem korisnik može upisati ime.

Kada korisnik refresh-a stranicu, ime mora ostati spremljeno.

Izazov 2: Spremite veličinu fonta

Dodajte mogućnost biranja veličine teksta:

small

medium

large

Vrijednost spremite u `localStorage` pomoću istog hooka.

Izazov 3: Napravite reset gumb

Dodajte gumb:

Reset preferences

Klikom na gumb tema se treba vratiti na `"light"`.

Kasnije možete proširiti hook tako da podržava i brisanje vrijednosti iz `localStoragea`.

Izazov 4: Koristite hook s objektom

Umjesto da imate više odvojenih vrijednosti, pokušajte spremiti objekt:

```
const [preferences, setPreferences] = useLocalStorage("preferences", {  
  theme: "light",  
  fontSize: "medium"  
});
```

Zatim mijenjajte samo jedno svojstvo objekta pomoću spread operatora.

Primjer:

```
setPreferences(current => ({  
  ...current,  
  theme: current.theme === "light" ? "dark" : "light"  
}));
```

Ovo pokazuje zašto u hooku koristimo `JSON.stringify` i `JSON.parse`. Hook može raditi i s objektima, ne samo sa stringovima.